

Analizator składów kosmetyków – „Inci Preferences”

Łucja Leśna i Oliwia Natzke

Spis treści

Wprowadzenie.....	3
Opis aplikacji.....	3
Geneza projektu.....	3
Cel aplikacji.....	3
Technologie.....	4
Backend: Django Framework.....	4
Baza danych: PostgreSQL.....	4
Frontend: HTML, CSS, JavaScript.....	4
Narzędzia.....	4
Deployment.....	4
Struktura projektu.....	5
Architektura.....	5
Moduł ingredients.....	5
Moduł cosmetics.....	5
Moduł preferences.....	5
Moduł users.....	6
Kategorie produktów.....	6
Funkcjonalność.....	7
Zarządzanie preferencjami składników.....	7
System rekomendacji.....	7
Wyszukiwanie zamienników.....	7
Przeglądanie kosmetyków.....	7
Szczegóły kosmetyku.....	8
Dodawanie kosmetyków.....	8
Ulubione kosmetyki.....	8
Zarządzanie kontem.....	8

Import danych.....	8
Zaawansowane wyszukiwanie.....	8
Skanowanie składu ze zdjęcia	9
Diagram przypadków użycia	10
Diagram klas	11
Kluczowe fragmenty kodu.....	12
Algorytm parsowania składników INCI.....	12
Algorytm wyszukiwania zamienników	13
Algorytm rekomendujący kosmetyki z największą liczbą bezpiecznych składników	14
Algorytm analizujący składki kosmetyków, które zostały dodane do ulubionych i na ich podstawie rekomendująca składniki, które najczęściej się pojawiają w tych kosmetykach, aby użytkownik mógł wygodnie zaznaczyć je jako bezpieczne.....	15
Baza danych.....	16
Relacje:	16
Schemat bazy danych:	16
Parsowanie składników:.....	17
Bezpieczeństwo:.....	17
Instrukcja użytkownika	18
Instalacja lokalna	18
Korzystanie z aplikacji.....	18
Zaawansowane funkcje.....	18
.....	19
Bezpieczeństwo.....	22
Hashowanie haseł.....	22
CSRF Protection	22
Session-based Authentication.....	22
Ochrona danych wrażliwych.....	22
Ograniczenia i przyszły rozwój.....	23
Obecne ograniczenia.....	23
Plany rozwoju.....	23

Wprowadzenie

Opis aplikacji

INCI Preferences to webowa aplikacja do analizy i zarządzania składami kosmetycznymi, zaprojektowana z myślą o użytkownikach poszukujących spersonalizowanego podejścia do wyboru produktów kosmetycznych.

Geneza projektu

Dostępne aplikacje i strony internetowe umożliwiają sprawdzanie składów kosmetyków, klasyfikując składniki jako korzystne lub niekorzystne dla skóry. Jednak istniejące rozwiązania mają wspólne ograniczenie – opierają swoje oceny na zasadzie uogólnienia, oznaczając składnik jako "dobry" lub "zły" w oparciu o jego działanie u większości użytkowników.

Takie podejście pomija indywidualną reakcję skóry, która może się znacząco różnić między osobami ze względu na typ skóry, schorzenia dermatologiczne, alergie czy stan bariery ochronnej. Osoba z wrażliwą skórą może źle reagować na składniki powszechnie uznawane za bezpieczne.

Projekt INCI Preferences umożliwia użytkownikom samodzielne określenie, które substancje są dla nich bezpieczne, które wymagają ostrożności, a których powinni unikać. Takie spersonalizowane podejście pozwala na stworzenie systemu rekomendacji dopasowanego do rzeczywistych, indywidualnych potrzeb.

Cel aplikacji

Głównym celem aplikacji jest umożliwienie użytkownikom:

- przeglądania bazy kosmetyków ze składami INCI,
- tworzenia profili, które umożliwiają ustawienie preferencji odnośnie danych składników,
- otrzymywania spersonalizowanych rekomendacji produktów,
- wyszukiwania zamienników kosmetyków o zbliżonym składzie.

Technologie

Backend: Django Framework

Wybrany ze względu na szybkość rozwoju, wbudowany panel administracyjny, gotowy system logowania użytkowników, oraz ułatwioną pracą z bazą danych bez pisania SQL.

Dodatkowo użyty został Django Rest Framework, który automatycznie przekształca dane z bazy na format JSON i oferuje przejrzystą dokumentację endpointów. Wykorzystane zostały również pomocnicze biblioteki: django-filter do filtrowania wyników, django-cors-headers do obsługi requestów z frontendu, oraz python-decouple do bezpiecznego przechowywania haseł do bazy danych.

Baza danych: PostgreSQL

PostgreSQL obsługuje złożone relacje many-to-many wymagane przez model danych aplikacji. Supabase zapewnia gotową infrastrukturę z connection poolingiem, automatycznymi backupami i możliwością skalowania.

Frontend: HTML, CSS, JavaScript

Czysty JavaScript bez frameworków zapewnia mniejszy rozmiar aplikacji, szybsze ładowanie oraz pełną kontrolę nad DOM. Dla prostej logiki frontendowej jest to wystarczające rozwiązanie.

Narzędzia

Git, Python venv oraz pip.

Deployment

Aplikacja przygotowana do wdrożenia na platformach wspierających Django. Baza danych na Supabase, pliki statyczne przez Whitenoise lub CDN. Aktualnie wdrożona na platformie Render pod adresem <https://inci-preferences.onrender.com>.

Struktura projektu

Architektura

Aplikacja Django została podzielona na cztery moduły:

1. **ingredients** - zarządzanie składnikami INCI
2. **cosmetics** - zarządzanie produktami
3. **preferences** - zarządzanie preferencjami użytkowników
4. **users** - zarządzanie kontami użytkowników

Moduł ingredients

Odpowiada za przechowywanie bazy składników kosmetycznych zgodnych z nomenklaturą INCI. Model zawiera nazwę składnika (inci_name), jego funkcję (purpose) oraz datę utworzenia. Moduł został zaprojektowany jako niezależna jednostka, co pozwala na rozbudowę bazy składników bez ingerencji w pozostałe części systemu. API oferuje endpoint do listowania składników wykorzystywany przez autocomplete w profilu użytkownika. System automatycznie tworzy nowe składniki podczas parsowania list INCI, oznaczając je jako purpose='Unknown' do późniejszego uzupełnienia.

Moduł cosmetics

stanowi centralny element aplikacji, obsługując produkty kosmetyczne oraz ich składniki. Model przechowuje podstawowe informacje (nazwa, marka), hierarchiczną kategoryzację (main_category → subcategory → product_type) oraz relację many-to-many ze składnikami. Kluczowa funkcjonalność to pole ingredients_text zawierające pełną listę INCI, które jest automatycznie parsowane metodą parse_and_add_ingredients() wykorzystującą regex do rozdzielania składników. Moduł implementuje algorytmy rekomendacji (wykluczanie produktów z niebezpiecznymi składnikami, sortowanie po bezpiecznych) oraz wyszukiwania zamienników (porównanie składów metodą podobieństwa Jaccarda). API oferuje kompleksowy zestaw endpointów do CRUD, filtrowania, wyszukiwania oraz zaawansowanych funkcji analitycznych.

Moduł preferences

Zarządza osobistymi preferencjami bezpieczeństwa składników oraz ulubionymi kosmetykami użytkowników. Model UserProfile zawiera trzy relacje many-to-many do składników (safe, moderate, unsafe) implementujące trójpoziomowy system klasyfikacji oraz relację do ulubionych kosmetyków. API umożliwia ustawianie preferencji składników, zarządzanie ulubionymi oraz pobieranie

spersonalizowanych list. Moduł zawiera również funkcję analizy ulubionych, która wykrywa często występujące składniki i sugeruje dodanie ich do preferencji.

Moduł users

Obsługuje pełny cykl życia konta użytkownika: rejestrację, logowanie, zarządzanie danymi oraz usuwanie konta. Wykorzystuje wbudowany system Django Auth rozszerzony o dedykowane widoki dostosowane do potrzeb aplikacji. System wymaga jedynie nazwy użytkownika i hasła (brak email), co upraszcza proces rejestracji. Użytkownicy mogą zmieniać swoją nazwę użytkownika, hasło oraz całkowicie usunąć konto wraz ze wszystkimi powiązanymi danymi.

Kategorie produktów

FACE

- Cleanser - produkty oczyszczające
- Moisturizer - kremy nawilżające
- Serum - serum do twarzy
- Essence - esencje
- Sunscreen - filtry przeciwsłoneczne

MAKEUP

- Eyes - produkty do makijażu oczu
 - Mascara, Eyeshadow, Eyeliner
- Face - produkty do makijażu twarzy
 - Foundation, Concealer, Powder, Blush, Bronzer, Highlighter
- Lips - produkty do makijażu ust
 - Lipstick, Lip Gloss
- Brows - produkty do brwi
 - Brow Pencil

BODY

- Produkty do ciała (kremy, żele, scrubs)

Funkcjonalność

Zarządzanie preferencjami składników

- Wyszukiwanie składników z autocomplete (includes, nie startsWith)
- Przypisywanie kolorów: zielony/żółty/czerwony
- Usuwanie składników z preferencji
- Wyświetlanie statystyk (liczba składników w każdej kategorii)
- Szybkie ustawianie preferencji przez kliknięcie składnika na stronie szczegółów kosmetyku (modal z trzema opcjami)
- Analiza ulubionych kosmetyków - automatyczne wykrywanie często występujących składników i sugerowanie dodania ich do safe (min 50% występowania w min 3 ulubionych)

System rekomendacji

- Wyświetlany na stronie głównej dla zalogowanych użytkowników
- Wyklucza kosmetyki z żółtymi lub czerwonymi składnikami
- Sortuje po liczbie zielonych składników (malejąco)
- Pokazuje top 10 rekomendacji
- Wyświetla liczbę dopasowanych safe składników

Wyszukiwanie zamienników

- Algorytm porównania składników: $\text{similarity} = (\text{matching} / \text{total}) * 100$
- Filtrowanie po tej samej kategorii głównej
- Minimalny próg podobieństwa: 40%
- Sortowanie od najbardziej podobnych
- Wyświetlanie procentu podobieństwa i liczby dopasowanych składników
- Top 10 wyników

Przeglądanie kosmetyków

- Kolorowe obramowania według bezpieczeństwa
- Hierarchiczna nawigacja po kategoriach (główna kategoria → podkategoria → typ produktu)

Szczegóły kosmetyku

- Kolorowe tagi składników według preferencji użytkownika
- Legenda kolorów określająca bezpieczeństwo składników (Safe/Moderate/Unsafe)
- Przycisk "Find Similar Products" prowadzący do wyszukiwania zamienników

Dodawanie kosmetyków

- Automatyczne parsowanie składników po zapisaniu

Ulubione kosmetyki

- Przycisk gwiazdki (☆) w prawym górnym rogu strony szczegółów
- Kolor limonkowy gdy nie dodany, różowy gdy w ulubionych
- Osobna strona z listą ulubionych

Zarządzanie kontem

- Rejestracja i logowanie (tylko username + password, bez email)
- Zmiana nazwy użytkownika
- Zmiana hasła (wymaga podania starego hasła)
- Usuwanie konta z potwierdzeniem hasłem

Import danych

- Import kosmetyków z pliku JSON
- Sprawdzanie duplikatów (name + brand)
- Automatyczne parsowanie składników podczas importu
- Raportowanie podsumowania (added/skipped/errors)

Zaawansowane wyszukiwanie

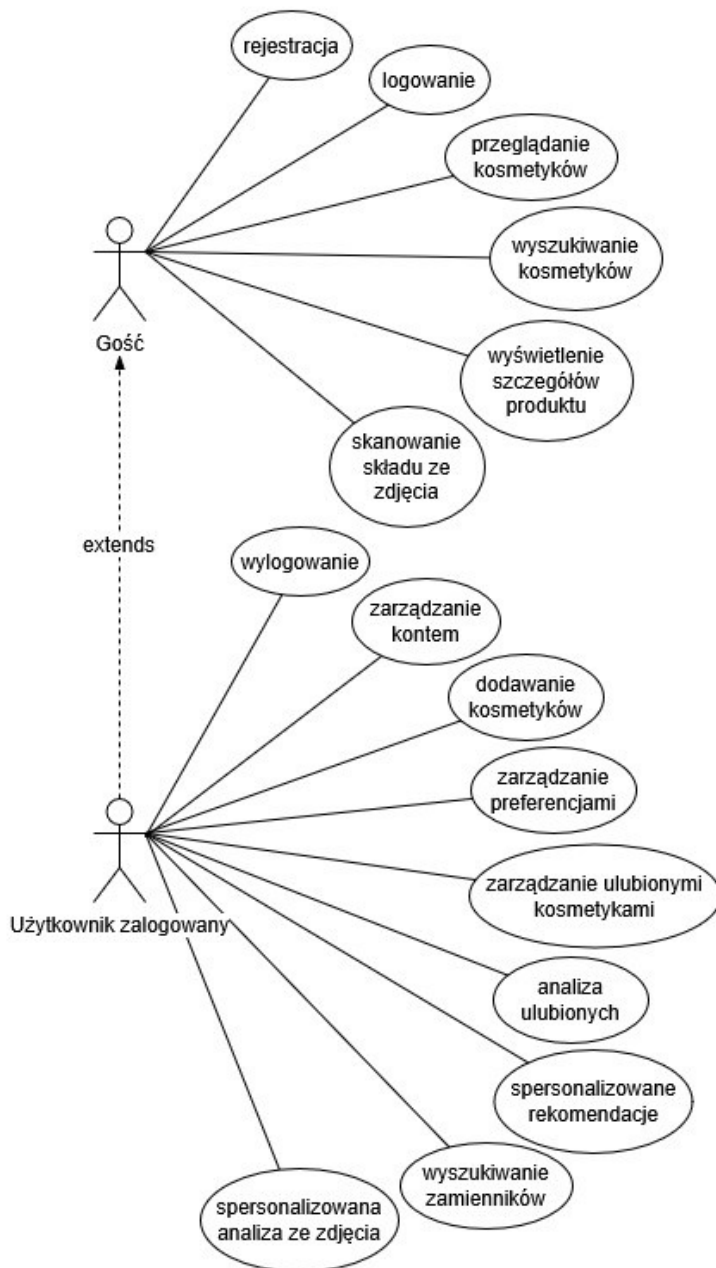
- Kaskadowe filtry kategorii (główna kategoria → podkategoria → typ produktu)
- Filtry bezpieczeństwa: All / Safe Only / Moderate & Safe (dla zalogowanych)
- Must Contain Ingredient - autocomplete z multi-select
- Must NOT Contain - wykluczanie składników z autocomplete

- Sortowanie: Name A-Z/Z-A, Brand, Safety (Best First), Most Safe Ingredients
- Aktywne filtry wyświetlane jako chipy nad wynikami
- Clear All Filters - resetowanie wszystkich filtrów jednym kliknięciem
- Zwijany panel filtrów

Skanowanie składu ze zdjęcia

- Upload zdjęcia produktu (click lub drag & drop)
- OCR (Tesseract.js) - rozpoznawanie tekstu ze zdjęcia
- Automatyczne parsowanie składników z rozpoznanego tekstu
- Wyświetlanie rozpoznanego tekstu z opcją edycji i ponownej analizy
- Analiza bezpieczeństwa według preferencji użytkownika
- Statystyki: liczby składników Safe/Moderate/Unsafe/Unknown
- Breakdown składników podzielony na kategorie z kolorowymi tagami
- Przycisk "Scan Another Product" do resetowania
- Obsługa formatów: JPG, PNG, HEIC

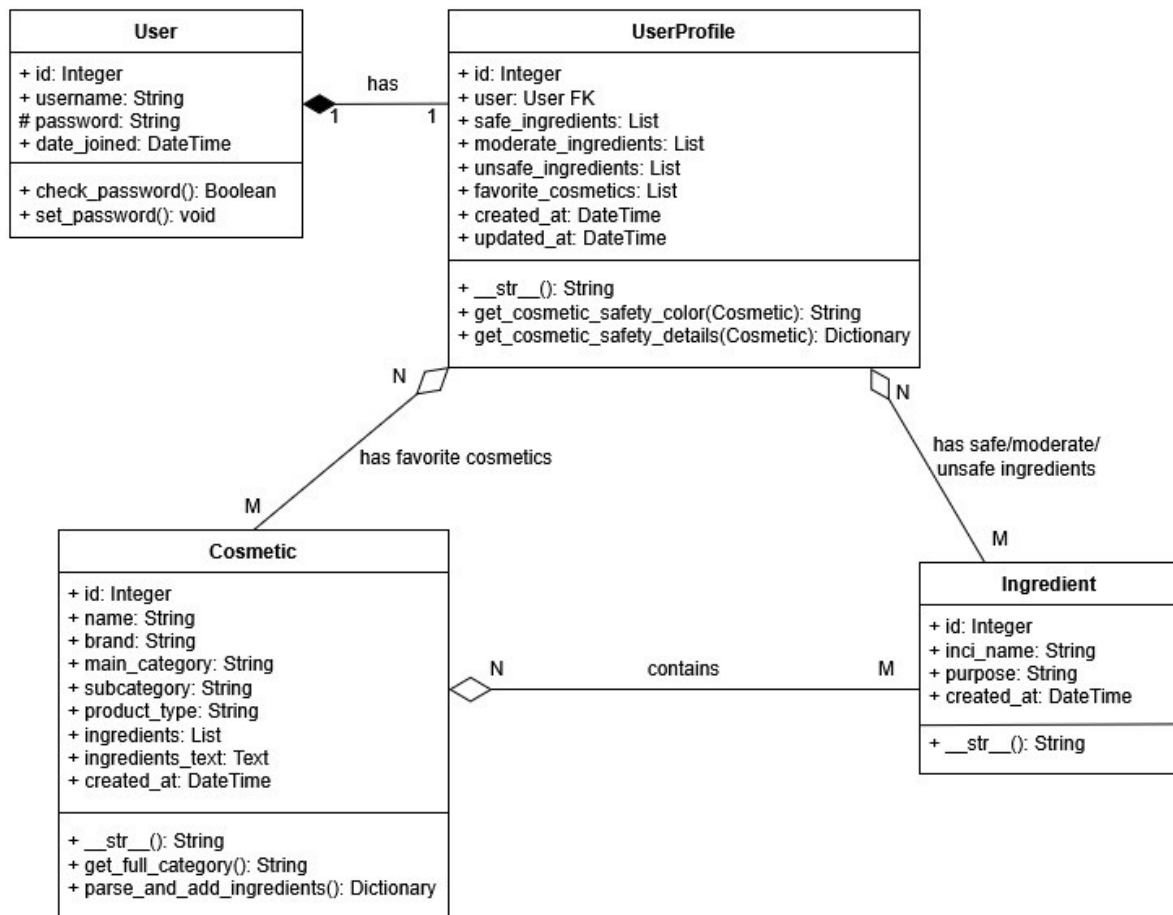
Diagram przypadków użycia



Rysunek 1. Diagram przypadków użycia aplikacji

Diagram przedstawia dwie role w systemie: Gość oraz Użytkownik zalogowany, który dziedziczy wszystkie uprawnienia gościa i dodatkowo uzyskuje dostęp do spersonalizowanych funkcji

Diagram klas



Rysunek 2. Diagram klas przedstawiający strukturę modeli bazy danych oraz relacje między nimi

Diagram przedstawia cztery główne klasy modeli aplikacji: **User** (wbudowany model Django), **UserProfile**, **Ingredient** oraz **Cosmetic**. Pomiędzy klasami zdefiniowano jedną relację jeden-do-jednego (**User**-**UserProfile**) oraz pięć relacji wiele-do-wielu, w tym trzy relacje preferencji składników (**safe**, **moderate**, **unsafe**), relację ulubionych kosmetyków oraz relację składu produktu.

Kluczowe fragmenty kodu

Algorytm parsowania składników INCI

apps/cosmetics/models.py

```
def parse_and_add_ingredients(self, auto_create=True):
    if not self.ingredients_text:
        return {'matched': [], 'not_found': []}

    # potrzebne do składników typu '1,2-Hexanediol'
    import re
    ingredients = re.split(r',\s+', self.ingredients_text)

    matched = []
    not_found = []

    for ingredient in ingredients:
        clean_name = ingredient.strip().strip('.')

        if not clean_name:
            continue

        # case-insensitive
        try:
            ingredient = Ingredient.objects.get(inci_name__iexact=clean_name)
            self.ingredients.add(ingredient)
            matched.append(ingredient.inci_name)
        except Ingredient.DoesNotExist:
            not_found.append(clean_name)
            if auto_create:
                ingredient = Ingredient.objects.create(
                    inci_name=clean_name,
                    purpose='Unknown'
                )
                self.ingredients.add(ingredient)
                matched.append(ingredient.inci_name)
            else:
                not_found.append(clean_name)

    return {
        'matched': matched,
        'not_found': not_found
    }
```

Funkcja `parse_and_add_ingredients()` odpowiada za automatyczne rozpoznawanie i przypisywanie składników INCI do kosmetyku na podstawie pola `ingredients_text`. Do podziału tekstu na pojedyncze składniki używany jest regex `r',\s+'`, który rozdziela po przecinku następowanym przez spację - dzięki temu składniki zawierające przecinek w nazwie (np. 1,2-Hexanediol) są obsługiwane poprawnie. Każdy składnik jest wyszukiwany w bazie danych w sposób case-insensitive. Jeśli składnik nie istnieje w bazie i parametr `auto_create` jest ustawiony na `True`, zostaje automatycznie utworzony z wartością `purpose='Unknown'`. Funkcja zwraca słownik z listą dopasowanych i nieznaalezionych składników.

Algorytm wyszukiwania zamienników

apps/cosmetics/views.py

```
@action(detail=True, methods=['get'])
def find_dupes(self, request, pk=None):
    original = self.get_object()
    original_ingredients = set(ing.id for ing in original.ingredients.all())

    if len(original_ingredients) == 0:
        return Response({'error': 'This cosmetic has no ingredients'}, status=400)

    dupes = []

    candidates = self.queryset.filter(
        main_category=original.main_category
    ).exclude(id=original.id)

    for cosmetic in candidates:
        cosmetic_ingredients = set(ing.id for ing in cosmetic.ingredients.all())

        if len(cosmetic_ingredients) == 0:
            continue

        matching = len(original_ingredients & cosmetic_ingredients)

        similarity = (matching / len(original_ingredients)) * 100

        # min 50% składników się pokrywa
        if similarity >= 50:
            dupes.append({
                'cosmetic': cosmetic,
                'similarity': round(similarity, 1),
                'matching': matching
            })

    dupes.sort(key=lambda x: x['similarity'], reverse=True)

    # top 10
    from apps.cosmetics.serializers import CosmeticSerializer
    result = [
        {
            **CosmeticSerializer(d['cosmetic']).data,
            'similarity_score': d['similarity'],
            'matching_ingredients_count': d['matching']
        }
        for d in dupes[:10]
    ]

    return Response({
        'original': CosmeticSerializer(original).data,
        'dupes': result
    })
```

Funkcja `find_dupes()` odpowiada za wyszukiwanie kosmetyków o podobnym składzie do wybranego produktu. Na początku pobierane są identyfikatory składników oryginalnego kosmetyku, a następnie jako kandydaci do porównania wybierane są wyłącznie produkty z tej samej kategorii głównej. Dla każdego kandydata obliczany jest współczynnik podobieństwa według wzoru: liczba wspólnych składników podzielona przez liczbę składników oryginału, pomnożona przez 100. Do wyznaczenia części wspólnej składników wykorzystywana jest operacja na zbiorach (&). Produkty z podobieństwem poniżej 50% są odrzucane, pozostałe sortowane malejąco i zwracane jako top 10 wyników wraz z procentowym podobieństwem i liczbą dopasowanych składników.

Algorytm rekomendujący kosmetyki z największą liczbą bezpiecznych składników

apps/cosmetics/views.py

```
@action(detail=False, methods=['get'], permission_classes=[IsAuthenticated])
def recommended(self, request):
    try:
        profile = request.user.profile
    except UserProfile.DoesNotExist:
        return Response({'error': 'No profile found'}, status=400)

    safe_ingredients = set(profile.safe_ingredients.all())
    moderate_ingredients = set(profile.moderate_ingredients.all())
    unsafe_ingredients = set(profile.unsafe_ingredients.all())

    recommendations = []

    for cosmetic in self.queryset.all():
        cosmetic_ingredients = set(cosmetic.ingredients.all())

        has_moderate = bool(cosmetic_ingredients.intersection(moderate_ingredients))
        has_unsafe = bool(cosmetic_ingredients.intersection(unsafe_ingredients))

        if has_moderate or has_unsafe:
            continue

        safe_count = len(cosmetic_ingredients.intersection(safe_ingredients))

        if safe_count > 0:
            recommendations.append({
                'cosmetic': cosmetic,
                'safe_count': safe_count
            })

    recommendations.sort(key=lambda x: x['safe_count'], reverse=True)

    top_recommendations = recommendations[:10]

    from apps.cosmetics.serializers import CosmeticSerializer
    serialized = [
        {
            **CosmeticSerializer(rec['cosmetic']).data,
            'safe_ingredients_count': rec['safe_count']
        }
        for rec in top_recommendations
    ]

    return Response({
        'count': len(serialized),
        'recommendations': serialized
    })
```

Funkcja `recommended()` generuje spersonalizowane rekomendacje kosmetyków na podstawie preferencji zalogowanego użytkownika. Na początku pobierane są trzy zbiory składników z profilu użytkownika: bezpieczne, umiarkowane i niebezpieczne. Następnie dla każdego kosmetyku w bazie sprawdzane jest, czy zawiera jakikolwiek składnik oznaczony jako umiarkowany lub niebezpieczny - jeśli tak, produkt jest pomijany. Dla pozostałych kosmetyków zliczana jest liczba składników bezpiecznych. Produkty z przynajmniej jednym bezpiecznym składnikiem są sortowane malejąco według tej liczby i zwracane jako top 10 rekomendacji wraz z informacją o liczbie dopasowanych bezpiecznych składników.

Algorytm analizujący składy kosmetyków, które zostały dodane do ulubionych i na ich podstawie rekomendująca składniki, które najczęściej się pojawiają w tych kosmetykach, aby użytkownik mógł wygodnie zaznaczyć je jako bezpieczne

apps/cosmetics/views.py

```
@action(detail=False, methods=['get'], permission_classes=[IsAuthenticated])
def analyze_favorites(self, request):
    try:
        profile = request.user.profile
    except:
        return Response({'error': 'No profile found'}, status=400)

    favorites = profile.favorite_cosmetics.all()

    if favorites.count() < 3:
        return Response({
            'message': 'Add at least 3 favorites to get ingredient insights',
            'suggestions': []
        })

    ingredient_counts = {}
    total_favorites = favorites.count()

    for cosmetic in favorites:
        for ingredient in cosmetic.ingredients.all():
            # pominięcie składników już oznaczone przez użytkownika
            if (ingredient.id in profile.safe_ingredients.all().values_list('id', flat=True) or
                ingredient.id in profile.moderate_ingredients.all().values_list('id', flat=True) or
                ingredient.id in profile.unsafe_ingredients.all().values_list('id', flat=True)):
                continue

            if ingredient.id not in ingredient_counts:
                ingredient_counts[ingredient.id] = {
                    'ingredient': ingredient,
                    'count': 0
                }
            ingredient_counts[ingredient.id]['count'] += 1

    # sortowanie po częstotliwości
    sorted_ingredients = sorted(
        ingredient_counts.values(),
        key=lambda x: x['count'],
        reverse=True
    )

    # top 5 składników które występują w min 50% ulubionych
    suggestions = []
    for item in sorted_ingredients[:10]:
        percentage = (item['count'] / total_favorites) * 100
        if percentage >= 50:
            suggestions.append({
                'ingredient_id': item['ingredient'].id,
                'ingredient_name': item['ingredient'].INCI_name,
                'count': item['count'],
                'percentage': round(percentage, 1),
                'total_favorites': total_favorites
            })

    return Response({
        'total_favorites': total_favorites,
        'suggestions': suggestions[:5] # max 5 sugestii
    })
```

Funkcja `analyze_favorites()` analizuje składy kosmetyków dodanych przez użytkownika do ulubionych i sugeruje składniki, które najczęściej się w nich pojawiają. Funkcja wymaga minimum 3 ulubionych kosmetyków, aby analiza miała sens. Dla każdego ulubionego kosmetyku zliczane są wystąpienia poszczególnych składników, przy czym pomijane są te, które użytkownik już wcześniej oznaczył jako bezpieczne, umiarkowane lub niebezpieczne. Następnie składniki są sortowane malejąco według liczby wystąpień i filtrowane - do sugestii trafiają tylko te, które występują w co najmniej 50% ulubionych kosmetyków. Zwracane jest maksymalnie 5 sugestii wraz z procentem występowania.

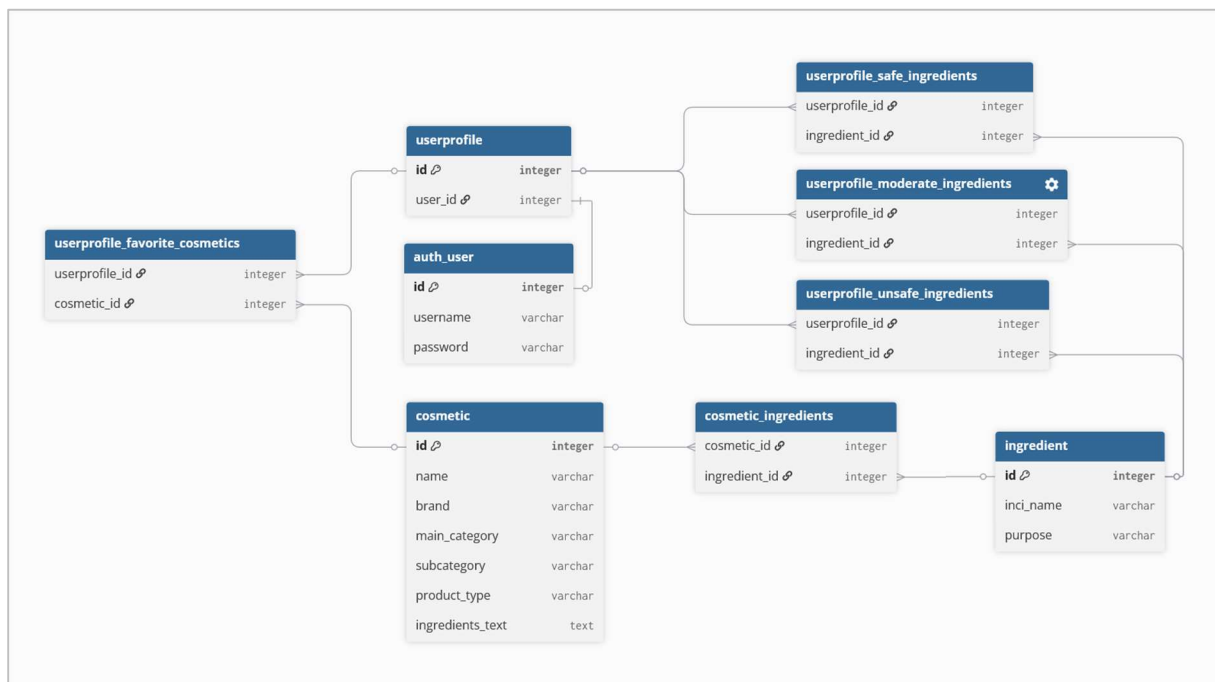
Baza danych

Serwer PostgreSQL bazy danych znajduje się na stronie supabase.com na serwerze AWS | eu-north-1.

Relacje:

- User 1:1 UserProfile
- Cosmetic M:N Ingredient (przez cosmetic_ingredients)
- UserProfile M:N Ingredient (safe_ingredients)
- UserProfile M:N Ingredient (moderate_ingredients)
- UserProfile M:N Ingredient (unsafe_ingredients)
- UserProfile M:N Cosmetic (favorite_cosmetics)

Schemat bazy danych:



Rysunek 3. Schemat bazy danych

Parsowanie składników:

- System automatycznie parsuje pole `ingredients_text` używając regex `r',\s+'` (przecinek + spacje)
- Metoda `parse_and_add_ingredients()` rozdziela składniki, wyszukuje je w bazie (case-insensitive) i tworzy relacje
- Nieistniejące składniki są automatycznie tworzone z `purpose='Unknown'`
- Składniki z przecinkami w nazwie (np. "1,2-Hexanediol") są obsługiwane poprawnie

Bezpieczeństwo:

- Hasła hashowane algorytmem PBKDF2 z SHA256 (domyślnie Django)
- Każde hasło ma unikalną sól
- Dane połączenia z bazą przechowywane w `.env`

Instrukcja użytkowania

Instalacja lokalna

- Sklonuj repozytorium: `git clone https://github.com/lulesna/analizator-skladow-kosmetykow`
- Utwórz wirtualne środowisko: `python -m venv venv`
- Aktywuj środowisko: `venv\Scripts\activate` (Windows) lub `source venv/bin/activate` (Linux/Mac)
- Zainstaluj zależności: `pip install -r requirements.txt`
- Utwórz plik `.env` w głównym katalogu z danymi do bazy Supabase
- Wykonaj migracje: `python manage.py migrate`
- Zaimportuj dane przykładowe: `python import_cosmetics_json.py`
- Uruchom serwer: `python manage.py runserver`
- Otwórz przeglądarkę: `http://localhost:8000`

Korzystanie z aplikacji

- Wejdź na stronę <https://inci-preferences.onrender.com/>
- Zarejestruj konto - kliknij "Register" w menu i podaj nazwę użytkownika oraz hasło
- Zaloguj się - kliknij "Login" i wprowadź swoje dane
- Ustaw preferencje składników - wejdź w "Profile", wyszukaj składniki i przypisz im kolory (zielony/żółty/czerwony)
- Przeglądaj kosmetyki - użyj wyszukiwarki lub przeglądaj po kategoriach
- Sprawdź szczegóły produktu - kliknij na produkt aby zobaczyć pełny skład z kolorowymi tagami
- Dodaj do ulubionych - kliknij gwiazdkę aby zapisać produkt
- Zobacz rekomendacje - na stronie głównej znajdziesz top 10 produktów dopasowanych do Twoich preferencji
- Skanuj produkty - użyj funkcji "Scan" aby przeanalizować skład ze zdjęcia opakowania

Zaawansowane funkcje

- Szybkie ustawianie preferencji - kliknij składnik na stronie szczegółów produktu, wybierz kolor z modala
- Analiza ulubionych - w profilu zobaczysz sugestie składników często występujących w Twoich ulubionych produktach
- Wyszukiwanie zamienników - kliknij "Find Similar Products" aby znaleźć produkty o podobnym składzie

- Zaawansowane filtrowanie - w wyszukiwarce użyj "Advanced Filters" dla precyzyjnego doboru produktów
- Dodawanie produktów - użyj formularza w "Add Cosmetic" aby dodać nowy kosmetyk do bazy

Scan Ingredients from Photo

Upload a photo of product ingredients and get instant analysis

Recognized Text

Aqua, Sodium Laureth Sulfate, Urea, Sodium Chloride, Cocamidopropyl Betaine, Glycerin, Coco-Glucoside, Glyceryl Oleate, Panthenol, Parfum, Hydrogenated Vegetable Glycerides Citrate, Tocopherol, Sodium Citrate, Citric Acid, Sodium Benzoate.

Edit Text

Safety Analysis

CAUTION! Contains 2 ingredients to watch

3
Safe

2
Moderate

0
Unsafe

6
Unknown

Ingredients Breakdown:

⚠ Moderate (2)

Sodium Chloride

Parfum

✅ Safe (3)

Aqua

Glycerin

Panthenol

Unknown (6)

Sodium Laureth Sulfate

Urea

Coco-Glucoside

Glyceryl Oleate

Sodium Citrate

Citric Acid

These ingredients are not in your preferences. Add them for more accurate analysis.

Scan Another Product

Rysunek 4. Widok strony skanowania składu po wgraniu zdjęcia produktu

Search for cosmetics

Search

▲ Advanced Filters

Category:

All
Face
Makeup
Body
Hands

Safety Level:

All
✓ Safe Only
⚠ Moderate & Safe

Must Contain Ingredient:

Start typing ingredient name...

Must NOT Contain:

Start typing ingredient name...

Sort By:

Name (A-Z) ▼

Clear All Filters
Apply Filters

Rysunek 5. Widok wyszukiwania kosmetyków z filtrowaniem

Skincare

Cleanser Moisturizer Sunscreen Serum Toner

All Skincare Products

Black Rice Moisture 5.5 Soft Cleansing Gel ▲

Brand: Haruharu Wonder

Type: Cleanser

Ingredients: 24

View Details

Birch Juice Moisturizing Sun Cream SPF50+ PA+++ ▲

Brand: Round Lab

Type: SPF

Ingredients: 40

View Details

PS Lipo-Active Moisturising Cream For Sensitive Or Dry Skin 🕒

Brand: Cetaphil

Type: Moisturizer

Ingredients: 25

View Details

Effaclar Gel Moussant Purifiant ▲

Hydrating Facial Cleanser ▲

Cicaplast Baume B5+ ✅

Rysunek 6. Widok przeglądania kosmetyków po kategoriach

Insights from Your Favorites

We noticed these ingredients appear frequently in your favorites:

Butylene Glycol

Appears in 5 of 6 favorites (83.3%)

[Add to Safe](#)

Ethylhexylglycerin

Appears in 4 of 6 favorites (66.7%)

[Add to Safe](#)

1,2-Hexanediol

Appears in 4 of 6 favorites (66.7%)

[Add to Safe](#)

Sodium Hyaluronate

Appears in 4 of 6 favorites (66.7%)

[Add to Safe](#)

Rysunek 7. Widok rekomendacji bezpiecznych składników na podstawie ulubionych kosmetyków

Find Similar Products

Hydro Boost Water Gel

Brand: Neutrogena

Ingredients: 8

Found 6 Similar Products

PS Lipo-Active Moisturising Cream For Sensitive Or Dry Skin

62.5% match

Brand: Cetaphil

Type: Moisturizer

Matching: 5/8

[View Details](#)

Daily Moisturizing Lotion

62.5% match

Brand: CeraVe

Type: Moisturizer

Matching: 5/8

[View Details](#)

Birch Juice Moisturizing Sun Cream SPF50+ PA++++

50% match

Brand: Round Lab

Type: Spf

Matching: 4/8

[View Details](#)

Rysunek 8. Widok polecenia zamienników danego kosmetyku

Bezpieczeństwo

Hashowanie haseł

Django domyślnie używa algorytmu PBKDF2 (Password-Based Key Derivation Function 2) z funkcją hash SHA256. Każde hasło otrzymuje unikalną, losowo wygenerowaną sól. Hasła nie są przechowywane w postaci jawnej - w bazie znajduje się tylko hash.

CSRF Protection

Wszystkie żądania POST wymagają tokena CSRF (Cross-Site Request Forgery). Token jest pobierany z cookies i dodawany do nagłówków requestów jako X-CSRFToken. Chroni to przed atakami gdzie złośliwa strona próbuje wykonać akcje w imieniu zalogowanego użytkownika.

Session-based Authentication

Aplikacja używa sesji zamiast tokenów JWT. Po zalogowaniu, Django tworzy sesję po stronie serwera i wysyła użytkownikowi cookie z session ID. Każde kolejne żądanie zawiera to cookie, co pozwala serwerowi zidentyfikować użytkownika. Sesje są bezpieczniejsze niż tokeny JWT, bo przechowywane server-side i mogą być natychmiast unieważnione.

Ochrona danych wrażliwych

- SECRET_KEY przechowywany w zmiennych środowiskowych, nigdy w kodzie
- Dane połączenia z bazą w pliku .env (dodany do .gitignore)
- HTTPS wymuszony przez Render.com
- ALLOWED_HOSTS ogranicza dozwolone domeny

Ograniczenia i przyszły rozwój

Obecne ograniczenia

- Brak pola ceny produktów - nie można porównywać cen w funkcji dupes
- Brak zdjęć produktów - kosmetyki identyfikowane tylko po nazwie i składzie
- OCR (skanowanie ze zdjęcia) wymaga dobrego oświetlenia i jakości zdjęcia
- Aplikacja dostępna tylko po angielsku
- Baza produktów wymaga ręcznego uzupełniania przez użytkowników

Plany rozwoju

- Dodanie pola price dla porównywania cen zamienników
- Upload zdjęć produktów
- System ocen i recenzji użytkowników
- Porównywarka side-by-side dla kilku produktów jednocześnie
- Wielojęzyczność